Computing for Analytical Work

Session 2 — What is running?

Block 3 — Python interpreters, packages, imports, and environments

Knowledge check 2 — 10 minutes, closed book, 3 questions

- Covers Session 1 and Homework 1: paths, working directory, terminal, `git status`
- Laptops closed. Paper only.
- Three short questions. Diagnostic, not tricky.
- Worth 5% of the final grade
- **This block's lecture is 35 minutes**, not 45 — the check comes out of lecture, never out of lab
- Hand it in when the timer goes. Then we begin.

This block in one sentence

"Python" means a particular interpreter with a particular set of installed packages.

Not a language. Not an icon. A specific program, on a specific path, with a specific list of packages it can see.

What we cover

- The reveal: what `uv run` has been doing all week
- Interpreter, script, notebook
- Packages, `import`, and `sys.path`
- Installing into a specific environment; why installs "fail"
- Many Pythons on one machine, and how to check which is active
- `uv`: the one workflow this program uses
- Git thread: `git add`, `git commit`
- AI thread, stretch, and Lab 3

You have been typing `uv run` for a week

```
uv run python scripts/summarize.py
```

You typed that in the setup check, in Lab 1, in Lab 2, in Homework 1.

The README said: *"You will learn what `uv run` does in Session 2. For now, type it exactly."*

It is Session 2. Here is what it was doing.

Interpreter, script, notebook

- **Interpreter** — a program, one executable file, that reads Python code and runs it
- **Script** — a `.py` file the interpreter reads top to bottom, once, then exits
- **Notebook** — a long-running interpreter (the *kernel*) you feed one cell at a time

```
uv run python -c "import sys; print(sys.executable)"
```

That prints the path of the interpreter that ran the line. It is a file. You can `ls` it.

What a package is

- A folder of Python code somebody else wrote, plus metadata
- `pandas`, `matplotlib`, `jupyter` — each is a folder on disk, installed *somewhere*
- Installed means: copied into a folder **that one interpreter knows to look in**

```
uv run python -c "import pandas; print(pandas.__file__)"
```

That prints where pandas lives. Note the path. It will matter in five slides.

What `import` actually does

```
import pandas
```

1. The interpreter takes a list of folders called `sys.path`
2. It walks that list, in order, looking for a folder or file named `pandas`
3. First match wins. No match: `ModuleNotFoundError`

`import` is a **search**. It is not a download, and it is not a question about your laptop.

`sys.path` belongs to *that* interpreter

```
uv run python -c "import sys; print('\n'.join(sys.path))"
```

- Every interpreter has its own list
- The list points at folders next to *that* interpreter
- A package installed next to Python A is invisible to Python B

So "is pandas installed?" is an incomplete question. **Installed for which interpreter?**

Installing into a specific environment

- An **environment** is one interpreter plus the folders it searches. That is all it is.
- Installing a package puts it into **one** environment
- This course: every project has its own environment, in `.venv/`, inside the project folder

```
uv add matplotlib
```

That installs matplotlib into *this project's* environment, and records that it did.

Why installs "fail"

The install did not fail. It **succeeded, somewhere else.**

- You typed `pip install matplotlib` in a terminal. It said *Successfully installed*.
- It installed into whichever Python that terminal's `pip` belonged to
- Your project's interpreter, or your notebook's kernel, has a different `sys.path`
- Result: `ModuleNotFoundError`, with a receipt saying it was installed

This is the single most common environment complaint in every course in this program.

Many Pythons on one machine

Typical laptop in this room, before this course:

- one Python from the operating system
- one from python.org, or Homebrew, or the Microsoft Store stub
- one inside Anaconda, if you ever installed it
- one per project, in `.venv/`, managed by `uv`

Each has its own `sys.path`. Nothing coordinates them. **Which one runs is decided by which one you invoke.**

How to check which Python is active

```
python --version              # which Python is on PATH right now
which python                  # where that Python actually lives
uv --version                  # confirm uv is installed
uv python list                # which Pythons uv knows about
uv run python -c "import sys; print(sys.executable)"   # the one inside the project env
```

Every one of these passes an argument to `python`. **Never type bare `python` in Git Bash** — it hangs, silently.

Two Pythons, side by side

```
python -c "import sys; print(sys.executable)"
uv run python -c "import sys; print(sys.executable)"
```

- Different paths? You have just seen the whole problem on your own screen.
- The first is "whatever the shell found." The second is *this project's* interpreter.
- If `sys.executable` contains `WindowsApps`, that is the Store stub. Disable it (setup notes, §Windows).

Part 2 — `uv`: the one workflow

The four commands

```
uv sync                          # install everything the project declares
uv add pandas                    # declare and install a new dependency
uv run python script.py          # run a script inside the project's environment
uv run jupyter lab               # launch JupyterLab inside the project's environment
```

Always from inside the project folder. That is the whole workflow for this program.

`pyproject.toml` and `uv.lock`, in plain words

- `pyproject.toml` — the project's **wish list**: "this project needs pandas and matplotlib"
- `uv.lock` — the **exact receipt**: every package, every version, that satisfied the wish list
- `uv add` edits both. `uv sync` reads both and makes `.venv/` match.
- Both files are committed. `.venv/` is not; it can be rebuilt from them anywhere.

```
cat pyproject.toml
```

So what was `uv run` doing?

```
uv run python scripts/summarize.py
```

1. Find the project folder (the one with `pyproject.toml`)
2. Make sure `.venv/` exists and matches `uv.lock` (a quiet `uv sync`)
3. Run `python scripts/summarize.py` **using the interpreter inside `.venv/`**

You never had to know which Python was on `PATH`. That was the point. Now you know why.

The notebook standard: the project's own kernel

In **VS Code** — the program's editor, and this course's:

1. Open the **project folder** (never the lone `.ipynb`)
2. Kernel picker (top right) → the interpreter inside `.venv/`
3. Verify: `import sys; sys.executable` in a cell → a path inside *this project's* `.venv/`

Alternative: `uv run jupyter lab` from the project folder — kernel pre-pointed, nothing to pick.

The kernel picker is where the mismatch lives

- Global Python, Anaconda, *another project's* `.venv` — all sit in the same picker, one click apart
- **One VS Code window, several Pythons:** the terminal, the notebook kernel, and the status-bar interpreter are chosen *separately*
- So `pip install` in the terminal can succeed — and the notebook still fails
- Result: `ModuleNotFoundError` for a package you installed ten minutes ago, with no warning

Same code, different interpreter, different truth. `sys.executable` — in *each* pane — settles it.

In the wild: `requirements.txt`

- Older repos, blog posts, and other courses ship `requirements.txt` instead of `pyproject.toml`
- Same job, older file: *make this project's packages available to its interpreter*
- `uv` handles it: `uv pip install -r requirements.txt`
- **Not lectured.** The Lab 3 stretch task and the reference in the README cover it.

If a README says `pip install -r requirements.txt` and there is no such file, that README is wrong.

A whiff of Git — save a checkpoint

```
git status          # what changed?
git add .           # stage it: "include these in the next snapshot"
git commit -m "Declare matplotlib in pyproject; fix README"
```

- A commit is a **checkpoint after a working fix**
- Commit when it works. Not before, not "at the end"

After you commit, `git diff` shows nothing

```
git commit -m "Declare matplotlib in pyproject; fix README"
git diff
```

- Empty output. **That is correct.** Your working tree now matches the snapshot.
- That is how you know the commit captured everything.
- If `git diff` still shows changes after `git commit`, something was left out. Look at `git status`.

AI thread — ask for evidence, not for a fix

Good questions for an assistant today:

- *"What evidence distinguishes 'package not installed' from 'installed in a different environment'?"*
- *"What does* `uv sync` *actually do? What is* `pyproject.toml` *? How is* `uv` *different from* `pip` *or* `conda` *?"*
- *"Here is* `sys.executable` *from my terminal and from my notebook. They differ. What does that mean?"*

Then run the inspection yourself and paste the real output.

The active interpreter decides what is true

- AI can suggest causes. It cannot see your `sys.path`.
- `uv run python -c "import sys; print(sys.executable)"` can.
- If the assistant says "it's installed" and the interpreter says `ModuleNotFoundError`, the interpreter is right.

Sources of truth today: the filesystem, the interpreter, `uv`'s output. Everything else is a hypothesis.

5-minute stand-and-stretch

Stand up. Leave the laptop. Back in five.

Lab 3 — It imports on my machine, but not here

First 15 minutes: no AI. Chat windows, AI editors, assistant tabs closed. Terminal, filesystem, and `uv` output only. Search engines are fine.

Why: every failure today is one an assistant diagnoses instantly. The reflex we want is *"check which interpreter,"* not *"ask what to check."* The exam has no chat window. Fifteen minutes is a floor on struggle, not a ceiling on speed.

TAs will ask you to close a window. That is not a penalty.

The symptom, and how to run it

```
git clone <your Lab 3 repo>
cd <the folder it created>
uv sync
uv run python scripts/plot.py
```

- It raises `ModuleNotFoundError: No module named 'matplotlib'`
- The README says `pip install -r requirements.txt`. There is no such file.
- Your job: find out which interpreter is looking, what the project *declares*, and why they disagree

What you produce, and the checkoff

1. `uv run python scripts/plot.py` runs clean, from a fresh `uv sync`
2. `pyproject.toml` declares the missing dependency (`uv add`, not a hand edit)
3. README setup instructions are correct for *this* repo
4. One commit of the working state; `git diff` is empty afterward
5. `DIAGNOSIS.md`: symptom, cause, evidence (paste real output), change, verification
6. Checkoff: explain it to a TA in about a minute. No notes.

Stretch: fresh clone + `uv sync`; then the `requirements.txt` reference in the README. Reset instructions are in the README; both resets destroy work.